
Checking Quran Recitation from Noisy ASR Transcripts: a Two-Task Competition on Real Learner Data

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We propose a competition on the checking problem behind every Quran-
2 memorization app: given a noisy ASR transcript of a learner reciting from memory,
3 decide *which* ayahs were recited, *where* each begins and ends, and *whether* the
4 learner should be corrected—without flagging the breath repeats, restarts and stut-
5 ters that make up most real recitation. We release 258 human-reviewed transcripts
6 of production recordings (1,267 gold ayah chunks) with two fixed scorecards, an
7 oracle-checked recording-level held-out split, a 10% review sample, checksums,
8 and three evaluated baselines, including the deployed app’s hand-built pipeline.

9 1 Social impact and task design

10 Hundreds of millions of Muslims memorize Quran, normally with a listener who corrects them. Apps
11 that supplement that listener transcribe the recitation with commercial ASR and must then solve a
12 *checking* problem on noisy text: isti’adhah and basmala, ASR errors, breath-catching repeats, full
13 restarts, stutters, self-corrections, skipped words, abandoned ayahs. A plain word-error diff calls all
14 of that a mistake. In our production data 87% of ayah chunks are clean, and over-flagging benign
15 behaviour destroys a learner’s trust faster than missing an occasional error. Public Quran speech
16 resources target ASR of professional read recitation; to our knowledge none labels the learner-facing
17 checking task on real memorization attempts.

- 18 • **Task A — detect + split.** Input: one raw transcript. Output: the set of ayah ids recited and a
19 segmentation of the words by ayah, or *abstain* for non-Quranic input. Frozen (v1.1).
- 20 • **Task B — within-ayah mistake events.** Input: the transcript words of *one* ayah (a gold Task A
21 chunk), the reference ayah text, and the chunk’s position in the recording. Output: a list of events,
22 each typed (restart_repeat, full_repeat, stutter, self_correction, substitution, omission, insertion,
23 word_order, truncation, asr_garble) with a verdict *benign* / *corrected* / *mistake* / *uncertain*—the
24 last because text alone cannot always separate ASR error from recitation error. Released as a
25 labeled pilot; full labels ship before launch (§4).

26 2 Dataset

27 **Source and consent.** 258 recordings by adult users of a Quran-memorization chat bot, under terms of
28 service that disclose review for service improvement; commercial ASR; every row human-reviewed
29 (ayah assignment, split, confidence) after LLM pre-labeling. 254 rows are Quranic (604 distinct ayahs
30 from 45 surahs; 1–46 ayahs and 2–261 words per recording, median 40), 4 are non-Quranic negatives.
31 Text only; learner and recording identifiers replaced by surrogate ids; negatives hand-checked for
32 personal content. The Quran is a closed public canon, so transcripts carry essentially no personal
33 information.

Split. Deterministic stratified 60/40 by *recording* (151/107 rows; 711/556 chunks), never by chunk, so one reciter’s ASR quirks cannot leak; negatives 2/2; known-ambiguous rows pinned to train so the held-out oracle floor is exactly 0. Task B shares the split and id space.

Task B labels. Frozen schema: verbatim hypothesis/reference words, script-derived token spans, verdict, note. Reproducibility rules: word-final letter drops are *uncertain*; a truncated *last* ayah of a recording is benign (the learner stopped recording), elsewhere a mistake; a slip inside an abandoned first attempt is swallowed by the repeat. The pilot covers 99 chunks of 20 recordings (86 clean; 17 events: 11 substitutions, 3 omissions, 2 truncations, 1 full repeat), LLM-proposed and marked human-pending until sign-off. A naive diff flags only 21% of all 1,267 chunks; the rest match the reference exactly after canonicalization, so completing the labels is a review of ≈ 270 chunks.

3 Metrics and baselines

Fixed scripts; lower is better; components in $[0, 1]$, always reported next to the scalar. Task A: $\text{research_score} = \text{detection err} + \text{split err} + \text{abstention err}$ (exact ayah-set match; word-assignment accuracy after metric-only orthographic canonicalization; abstention on negatives). Task B: $\text{study3_score} = 2 \cdot \text{miss} + \text{false_alarm} + \text{benign} + \text{clean}$. A prediction matches a gold event if their token spans overlap. *miss*: gold mistakes not flagged (an *uncertain* prediction on one costs a half); *false_alarm*: flags touching no gold mistake/uncertain event, over all flags; *benign*: gold benign/corrected events flagged as mistakes; *clean*: clean chunks with any flag—the product’s false-flag rate, its own term because clean is the majority class. Adjacent same-type predictions are merged, so word- and phrase-level detectors score alike; type agreement and uncertain-rate are reported, not summed. Both scores: empty system 2.0, gold 0.

Task	Baseline	train	test	test components	
A	empty stub	2.000	2.000	det 1, split 1	abstain 0
A	incumbent production pipeline (hand-built)	0.785 [†]	0.760	det 0.17, split 0.09	abstain 0.50
A	coding-agent loop, best of 6 runs ¹	0.008 [†]	0.079	det 0.06, split 0.02	abstain 0
B	B0 empty stub	2.000	2.000	miss 1	
B	B1 naive diff (every edit is a mistake)	1.358	1.111	benign 1	false_alarm 0.11
B	B2 incumbent cleaner + DP alignment	0.167	1.111	benign 1	false_alarm 0.11

Table 1: Baselines. [†]full 258-row set. Task B: pilot labels (47 train / 52 test chunks); both existing designs flag the held-out full repeat as a mistake (*benign* = 1).

On Task A, six autonomous runs of two coding agents in a Karpathy-style fixed-metric loop all matched or beat the incumbent held-out, the best by $10\times^1$; without a held-out split that loop memorized rows—hence the hidden final test below. The Task B pilot is small by design: it fixes schema and metric and already separates the failure modes that matter.

4 Competition plan, feasibility and ethics

Phases. (1) Launch with Task A frozen and Task B fully human-reviewed on the 258 recordings (≈ 270 -chunk review; tooling exists). (2) Public leaderboard on the released test split. (3) Final ranking on a *hidden* test set of new recordings drawn from $>20k$ further production recordings of the same app, labeled with the same protocol, user-reported false flags over-sampled as adversarial gold; released afterwards. A submission is one Python file per contract; scoring takes under a second, so an *agent track* (autonomous loops, logged per experiment) runs beside the human track under identical rules.

Feasibility. Harness, data, checksums and the 10% sample (26 recordings, both tasks) are attached; baselines run offline. **Ethics.** Text only, de-identified, adult users, ToS-disclosed review, CC BY 4.0; the task is corrective feedback, not surveillance, and *uncertain* is a first-class output so systems are not pushed to over-claim. **Participation.** Pure text, tiny data, sub-second evaluation, a task every Quran-learning app faces; Arabic NLP, speech and coding-agent groups are natural entrants.

¹Autoresearch with coding agents: generalizers and metric-maximizers on Quran recitation data, AIST 2026 / arXiv; citation withheld for double-blind review.